



భారతీయ సాంకేతిక విజ్ఞాన సంస్థ హైదరాబాద్
भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

DSP Lab - EE2801

Experiment-5

BY

Muthyala Koushik EE24BTECH11044

2026-04-06

Department of Electrical Engineering Indian Institute of Technology Hyderabad

Contents

1	Objective	1
2	Theory	2
3	Verilog Implementation	2
4	Resource Utilization	14
5	Output Comparison	22
6	FIR Filter Using IP Core	25

1 Objective

1. Load Intel Quartus prime lite software and modelsim into your laptop
2. Repeat experiment 4 with pipeline architecture , in Verilog. do for all 3 methods
3. Submit a report on resource utilisation for both non pipeline and pipeline approach
4. Using FIR IP, repeat experiment 4 and submit your report on comparison between IPs and user defined modules
5. Submit code and required reports for all the cases

2 Theory

FIR Filter

A Finite Impulse Response (FIR) filter is a digital filter widely used in digital signal processing applications. FIR filters are stable and can be designed to have a linear phase response. The output of an FIR filter is obtained by multiplying input samples with filter coefficients and summing the results.

$$y[n] = \sum_{k=0}^{N-1} h[k] \cdot x[n - k]$$

where $x[n]$ is the input signal, $h[k]$ are the filter coefficients, and $y[n]$ is the output signal.

Pipeline Architecture

Pipeline architecture is used to increase the speed of digital circuits by dividing the computation into multiple stages separated by registers. In FIR filters, registers are inserted between arithmetic operations to reduce the critical path delay and allow higher clock frequencies.

Implementation Methods

In this work, the FIR filter is implemented in Verilog using pipeline architecture through three methods: Direct form, Optimized form, and Generate block (genvar) implementation. Each method represents a different coding approach for describing the filter structure.

Resource Utilization

Resource utilization indicates the amount of FPGA hardware resources used in the design. The synthesis reports generated by Quartus provide information about logic elements, registers, and other resources used. These reports are used to compare pipelined and non-pipelined implementations.

3 Verilog Implementation

Direct Form Pipeline Implementation

Verilog Code

```
1 module direct_pipeline (
2     input wire      clk,
3     input wire      rst,
```

```

4      input wire signed [15:0] x_in,
5      output reg signed [39:0] y_out
6  );
7
8  parameter TAPS = 100;
9
10 wire signed [15:0] h      [0:TAPS-1]; // filter coefficients (Q2.14)
11 reg signed [15:0] x_pipe  [0:TAPS-2][0:1]; // two regs between each stage (
    top rail)
12 reg signed [39:0] sum_pipe [0:TAPS-2];      // one sum reg between each stage
    (bottom rail)
13 reg signed [39:0] mul_reg  [0:TAPS-1];      // registered multiplier outputs
14
15 // Filter coefficients (Q2.14 format)
16 assign h[0]   = -16'sd3;
17 assign h[1]   = -16'sd7;
18 assign h[2]   = -16'sd9;
19 assign h[3]   = -16'sd8;
20 assign h[4]   = -16'sd3;
21 assign h[5]   = 16'sd4;
22 assign h[6]   = 16'sd11;
23 assign h[7]   = 16'sd15;
24 assign h[8]   = 16'sd14;
25 assign h[9]   = 16'sd6;
26 assign h[10]  = -16'sd7;
27 assign h[11]  = -16'sd21;
28 assign h[12]  = -16'sd29;
29 assign h[13]  = -16'sd26;
30 assign h[14]  = -16'sd11;
31 assign h[15]  = 16'sd13;
32 assign h[16]  = 16'sd38;
33 assign h[17]  = 16'sd52;
34 assign h[18]  = 16'sd47;
35 assign h[19]  = 16'sd20;
36 assign h[20]  = -16'sd22;
37 assign h[21]  = -16'sd64;
38 assign h[22]  = -16'sd87;
39 assign h[23]  = -16'sd78;
40 assign h[24]  = -16'sd33;
41 assign h[25]  = 16'sd36;

```

```
42 assign h[26] = 16'sd104;
43 assign h[27] = 16'sd141;
44 assign h[28] = 16'sd125;
45 assign h[29] = 16'sd52;
46 assign h[30] = -16'sd57;
47 assign h[31] = -16'sd164;
48 assign h[32] = -16'sd222;
49 assign h[33] = -16'sd197;
50 assign h[34] = -16'sd83;
51 assign h[35] = 16'sd91;
52 assign h[36] = 16'sd263;
53 assign h[37] = 16'sd360;
54 assign h[38] = 16'sd324;
55 assign h[39] = 16'sd139;
56 assign h[40] = -16'sd156;
57 assign h[41] = -16'sd465;
58 assign h[42] = -16'sd661;
59 assign h[43] = -16'sd625;
60 assign h[44] = -16'sd285;
61 assign h[45] = 16'sd352;
62 assign h[46] = 16'sd1194;
63 assign h[47] = 16'sd2077;
64 assign h[48] = 16'sd2811;
65 assign h[49] = 16'sd3227;
66 assign h[50] = 16'sd3227;
67 assign h[51] = 16'sd2811;
68 assign h[52] = 16'sd2077;
69 assign h[53] = 16'sd1194;
70 assign h[54] = 16'sd352;
71 assign h[55] = -16'sd285;
72 assign h[56] = -16'sd625;
73 assign h[57] = -16'sd661;
74 assign h[58] = -16'sd465;
75 assign h[59] = -16'sd156;
76 assign h[60] = 16'sd139;
77 assign h[61] = 16'sd324;
78 assign h[62] = 16'sd360;
79 assign h[63] = 16'sd263;
80 assign h[64] = 16'sd91;
81 assign h[65] = -16'sd83;
```

```

82 assign h[66] = -16'sd197;
83 assign h[67] = -16'sd222;
84 assign h[68] = -16'sd164;
85 assign h[69] = -16'sd57;
86 assign h[70] = 16'sd52;
87 assign h[71] = 16'sd125;
88 assign h[72] = 16'sd141;
89 assign h[73] = 16'sd104;
90 assign h[74] = 16'sd36;
91 assign h[75] = -16'sd33;
92 assign h[76] = -16'sd78;
93 assign h[77] = -16'sd87;
94 assign h[78] = -16'sd64;
95 assign h[79] = -16'sd22;
96 assign h[80] = 16'sd20;
97 assign h[81] = 16'sd47;
98 assign h[82] = 16'sd52;
99 assign h[83] = 16'sd38;
100 assign h[84] = 16'sd13;
101 assign h[85] = -16'sd11;
102 assign h[86] = -16'sd26;
103 assign h[87] = -16'sd29;
104 assign h[88] = -16'sd21;
105 assign h[89] = -16'sd7;
106 assign h[90] = 16'sd6;
107 assign h[91] = 16'sd14;
108 assign h[92] = 16'sd15;
109 assign h[93] = 16'sd11;
110 assign h[94] = 16'sd4;
111 assign h[95] = -16'sd3;
112 assign h[96] = -16'sd8;
113 assign h[97] = -16'sd9;
114 assign h[98] = -16'sd7;
115 assign h[99] = -16'sd3;
116
117 integer i;
118
119 always @(posedge clk or posedge rst) begin
120     if (rst) begin
121         for (i = 0; i < TAPS; i = i + 1)
    
```

```

122         mul_reg[i] <= 40'sd0;
123     for (i = 0; i < TAPS-1; i = i + 1) begin
124         x_pipe[i][0] <= 16'sd0;
125         x_pipe[i][1] <= 16'sd0;
126         sum_pipe[i] <= 40'sd0;
127     end
128     y_out <= 40'sd0;
129 end
130 else begin
131     // Multipliers
132     mul_reg[0] <= x_in * h[0]; // stage 0: x_in feeds directly, no input
133                                // register
134     for (i = 1; i < TAPS; i = i + 1)
135         mul_reg[i] <= x_pipe[i-1][1] * h[i];
136
137     // Shift register (top rail)
138     x_pipe[0][0] <= x_in;
139     x_pipe[0][1] <= x_pipe[0][0];
140     for (i = 1; i < TAPS-1; i = i + 1) begin
141         x_pipe[i][0] <= x_pipe[i-1][1];
142         x_pipe[i][1] <= x_pipe[i][0];
143     end
144
145     // Accumulator pipeline (bottom rail)
146     sum_pipe[0] <= mul_reg[0];
147     for (i = 1; i < TAPS-1; i = i + 1)
148         sum_pipe[i] <= sum_pipe[i-1] + mul_reg[i];
149
150     // Register output
151     y_out <= sum_pipe[TAPS-2] + mul_reg[TAPS-1];
152 end
153
154 endmodule

```

Explanation

This module implements a 100-tap FIR filter using a direct form pipelined structure. Each input sample is multiplied with the filter coefficients and the products are accumulated to produce the output. Pipeline registers are inserted between stages to reduce the critical path delay and allow higher clock

frequency operation. Because the filter has 100 taps, the output appears after approximately 100 clock cycles of latency.

Optimized Pipeline Implementation

Verilog Code

```

1 module optimized_pipeline (
2     input wire          clk,
3     input wire          rst,
4     input wire signed [15:0] x_in,
5     output reg signed [39:0] y_out
6 );
7
8 parameter TAPS = 100;
9 parameter HALF = TAPS / 2;
10
11 wire signed [15:0] h [0:HALF-1];
12
13 // Coefficients
14 assign h[0]  = -16'sd3;    assign h[1]  = -16'sd7;    assign h[2]  = -16'sd9;
15 assign h[3]  = -16'sd8;    assign h[4]  = -16'sd3;    assign h[5]  = 16'sd4;
16 assign h[6]  = 16'sd11;    assign h[7]  = 16'sd15;    assign h[8]  = 16'sd14;
17 assign h[9]  = 16'sd6;     assign h[10] = -16'sd7;    assign h[11] = -16'sd21;
18 assign h[12] = -16'sd29;    assign h[13] = -16'sd26;    assign h[14] = -16'sd11;
19 assign h[15] = 16'sd13;     assign h[16] = 16'sd38;    assign h[17] = 16'sd52;
20 assign h[18] = 16'sd47;     assign h[19] = 16'sd20;    assign h[20] = -16'sd22;
21 assign h[21] = -16'sd64;    assign h[22] = -16'sd87;    assign h[23] = -16'sd78;
22 assign h[24] = -16'sd33;    assign h[25] = 16'sd36;    assign h[26] = 16'sd104;
23 assign h[27] = 16'sd141;    assign h[28] = 16'sd125;    assign h[29] = 16'sd52;
24 assign h[30] = -16'sd57;    assign h[31] = -16'sd164;    assign h[32] = -16'sd222;
25 assign h[33] = -16'sd197;    assign h[34] = -16'sd83;    assign h[35] = 16'sd91;
26 assign h[36] = 16'sd263;    assign h[37] = 16'sd360;    assign h[38] = 16'sd324;
27 assign h[39] = 16'sd139;    assign h[40] = -16'sd156;    assign h[41] = -16'sd465;
28 assign h[42] = -16'sd661;    assign h[43] = -16'sd625;    assign h[44] = -16'sd285;
29 assign h[45] = 16'sd352;    assign h[46] = 16'sd1194;    assign h[47] = 16'sd2077
30     ;
31 assign h[48] = 16'sd2811;    assign h[49] = 16'sd3227;
32
33 // Registers

```

```

33 reg signed [15:0] x_delay [0:TAPS-1];
34 reg signed [16:0] sd [0:HALF-1][0:HALF-2];
35 reg signed [39:0] mul_reg [0:HALF-1];
36 reg signed [39:0] sum_pipe [0:HALF-2];
37
38 integer i, j;
39
40 always @(posedge clk or posedge rst) begin
41     if (rst) begin
42         // Reset delay line
43         for (i = 0; i < TAPS; i = i + 1)
44             x_delay[i] <= 16'sd0;
45
46         // Reset FIR pipeline
47         for (i = 0; i < HALF; i = i + 1) begin
48             mul_reg[i] <= 40'sd0;
49             for (j = 0; j < HALF-1; j = j + 1)
50                 sd[i][j] <= 17'sd0;
51         end
52
53         for (i = 0; i < HALF-1; i = i + 1)
54             sum_pipe[i] <= 40'sd0;
55
56         y_out <= 40'sd0;
57     end
58     else begin
59         // Shift register
60         for (i = TAPS-1; i > 0; i = i - 1)
61             x_delay[i] <= x_delay[i-1];
62         x_delay[0] <= x_in;
63
64         // Stage 0
65         mul_reg[0] <= (x_delay[0] + x_delay[TAPS-1]) * h[0];
66
67         // Stages 1..HALF-1
68         for (i = 1; i < HALF; i = i + 1) begin
69             sd[i][0] <= x_delay[i] + x_delay[TAPS-1-i];
70             for (j = 1; j < i; j = j + 1)
71                 sd[i][j] <= sd[i][j-1];
72             mul_reg[i] <= sd[i][i-1] * h[i];

```

```

73     end
74
75     // Accumulator pipeline
76     sum_pipe[0] <= mul_reg[0];
77     for (i = 1; i < HALF-1; i = i + 1)
78         sum_pipe[i] <= sum_pipe[i-1] + mul_reg[i];
79
80     // Output
81     y_out <= sum_pipe[HALF-2] + mul_reg[HALF-1];
82 end
83 end
84
85 endmodule

```

Explanation

This design implements a pipelined FIR filter using coefficient symmetry. Pairs of input samples are added before multiplication, which reduces the number of multipliers to half of the total taps. Pipeline registers are used to maintain high operating speed while performing the accumulation. The output is generated after the pipeline latency.

Genvar Pipeline Implementation

Verilog Code

```

1 module genvar_pipeline (
2     input wire          clk,
3     input wire          rst,
4     input wire signed [15:0] x_in,
5     output wire signed [39:0] y_out
6 );
7
8 parameter TAPS = 100;
9
10 // Filter coefficients (Q2.14 format)
11 wire signed [15:0] h [0:TAPS-1];
12
13 assign h[0]  = -16'sd3;
14 assign h[1]  = -16'sd7;
15 assign h[2]  = -16'sd9;

```

```
16 assign h[3] = -16'sd8;
17 assign h[4] = -16'sd3;
18 assign h[5] = 16'sd4;
19 assign h[6] = 16'sd11;
20 assign h[7] = 16'sd15;
21 assign h[8] = 16'sd14;
22 assign h[9] = 16'sd6;
23 assign h[10] = -16'sd7;
24 assign h[11] = -16'sd21;
25 assign h[12] = -16'sd29;
26 assign h[13] = -16'sd26;
27 assign h[14] = -16'sd11;
28 assign h[15] = 16'sd13;
29 assign h[16] = 16'sd38;
30 assign h[17] = 16'sd52;
31 assign h[18] = 16'sd47;
32 assign h[19] = 16'sd20;
33 assign h[20] = -16'sd22;
34 assign h[21] = -16'sd64;
35 assign h[22] = -16'sd87;
36 assign h[23] = -16'sd78;
37 assign h[24] = -16'sd33;
38 assign h[25] = 16'sd36;
39 assign h[26] = 16'sd104;
40 assign h[27] = 16'sd141;
41 assign h[28] = 16'sd125;
42 assign h[29] = 16'sd52;
43 assign h[30] = -16'sd57;
44 assign h[31] = -16'sd164;
45 assign h[32] = -16'sd222;
46 assign h[33] = -16'sd197;
47 assign h[34] = -16'sd83;
48 assign h[35] = 16'sd91;
49 assign h[36] = 16'sd263;
50 assign h[37] = 16'sd360;
51 assign h[38] = 16'sd324;
52 assign h[39] = 16'sd139;
53 assign h[40] = -16'sd156;
54 assign h[41] = -16'sd465;
55 assign h[42] = -16'sd661;
```

```
56 assign h[43] = -16'sd625;
57 assign h[44] = -16'sd285;
58 assign h[45] = 16'sd352;
59 assign h[46] = 16'sd1194;
60 assign h[47] = 16'sd2077;
61 assign h[48] = 16'sd2811;
62 assign h[49] = 16'sd3227;
63 assign h[50] = 16'sd3227;
64 assign h[51] = 16'sd2811;
65 assign h[52] = 16'sd2077;
66 assign h[53] = 16'sd1194;
67 assign h[54] = 16'sd352;
68 assign h[55] = -16'sd285;
69 assign h[56] = -16'sd625;
70 assign h[57] = -16'sd661;
71 assign h[58] = -16'sd465;
72 assign h[59] = -16'sd156;
73 assign h[60] = 16'sd139;
74 assign h[61] = 16'sd324;
75 assign h[62] = 16'sd360;
76 assign h[63] = 16'sd263;
77 assign h[64] = 16'sd91;
78 assign h[65] = -16'sd83;
79 assign h[66] = -16'sd197;
80 assign h[67] = -16'sd222;
81 assign h[68] = -16'sd164;
82 assign h[69] = -16'sd57;
83 assign h[70] = 16'sd52;
84 assign h[71] = 16'sd125;
85 assign h[72] = 16'sd141;
86 assign h[73] = 16'sd104;
87 assign h[74] = 16'sd36;
88 assign h[75] = -16'sd33;
89 assign h[76] = -16'sd78;
90 assign h[77] = -16'sd87;
91 assign h[78] = -16'sd64;
92 assign h[79] = -16'sd22;
93 assign h[80] = 16'sd20;
94 assign h[81] = 16'sd47;
95 assign h[82] = 16'sd52;
```

```

96 assign h[83] = 16'sd38;
97 assign h[84] = 16'sd13;
98 assign h[85] = -16'sd11;
99 assign h[86] = -16'sd26;
100 assign h[87] = -16'sd29;
101 assign h[88] = -16'sd21;
102 assign h[89] = -16'sd7;
103 assign h[90] = 16'sd6;
104 assign h[91] = 16'sd14;
105 assign h[92] = 16'sd15;
106 assign h[93] = 16'sd11;
107 assign h[94] = 16'sd4;
108 assign h[95] = -16'sd3;
109 assign h[96] = -16'sd8;
110 assign h[97] = -16'sd9;
111 assign h[98] = -16'sd7;
112 assign h[99] = -16'sd3;
113
114 // a_chain[0:TAPS] and b_chain[0:TAPS]: one extra entry at index 0 holds the
    inputs
115 // stage k reads a_chain[k], b_chain[k] and writes to a_chain[k+1], b_chain[k
    +1]
116 // no k-1 indexing anywhere eliminates the a_chain[-1] warning
117 wire signed [15:0] a_chain [0:TAPS];
118 wire signed [39:0] b_chain [0:TAPS];
119
120 // Index 0: feed x_in and 0 directly into the first stage
121 assign a_chain[0] = x_in;
122 assign b_chain[0] = 40'sd0;
123
124 genvar k;
125 generate
126     for (k = 0; k < TAPS; k = k + 1) begin : fir_stage
127         fir_ffb u_ffb (
128             .clk    (clk),
129             .rst    (rst),
130             .a_in   (a_chain[k]),
131             .b_in   (b_chain[k]),
132             .h_i    (h[k]),
133             .a_out  (a_chain[k+1]),

```

```

134         .b_out (b_chain[k+1])
135     );
136     end
137 endgenerate
138
139 assign y_out = b_chain[TAPS];
140
141 endmodule
142
143
144 // fir_ffb: matches fir_pipeline stage exactly
145 // Top rail Reg 1: a1      <= a_in
146 // Top rail Reg 2: a_out   <= a1      (feeds next stage via a_chain)
147 // Multiply reg: mul_reg <= a_in * h_i
148 // Sum reg:      b_out    <= b_in + mul_reg
149 module fir_ffb (
150     input wire          clk,
151     input wire          rst,
152     input wire signed [15:0] a_in,
153     input wire signed [39:0] b_in,
154     input wire signed [15:0] h_i,
155     output reg signed [15:0] a_out,
156     output reg signed [39:0] b_out
157 );
158
159     reg signed [15:0] a1;      // top-rail register 1
160     reg signed [39:0] mul_reg; // registered multiplier output
161
162     always @(posedge clk or posedge rst) begin
163         if (rst) begin
164             a1      <= 16'sd0;
165             a_out    <= 16'sd0;
166             mul_reg  <= 40'sd0;
167             b_out    <= 40'sd0;
168         end
169         else begin
170             a1      <= a_in;          // top-rail reg 1
171             a_out    <= a1;          // top-rail reg 2 (feeds next stage)
172             mul_reg  <= a_in * h_i;   // registered multiply
173             b_out    <= b_in + mul_reg; // registered accumulate

```

```
174     end
175   end
176
177 endmodule
```

Explanation

This implementation uses the `genvar` and `generate` constructs in Verilog to create multiple FIR filter stages automatically. Each stage performs multiplication with the filter coefficient and accumulation of the partial sum. A small module representing a single stage is instantiated repeatedly using a `generate` loop. This method reduces code repetition and makes the design scalable for large numbers of taps while maintaining pipeline operation.

4 Resource Utilization

Compilation Reports

Flow Summary	
<<Filter>>	
Flow Status	Successful - Mon Apr 6 14:18:07 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	direct
Top-level Entity Name	direct
Family	Cyclone V
Device	5CGXFC7C7F23C8
Timing Models	Final
Logic utilization (in ALMs)	1,132 / 56,480 (2 %)
Total registers	1902
Total pins	58 / 268 (22 %)
Total virtual pins	0
Total block memory bits	0 / 7,024,640 (0 %)
Total DSP Blocks	99 / 156 (63 %)
Total HSSI RX PCSs	0 / 6 (0 %)
Total HSSI PMA RX Deserializers	0 / 6 (0 %)
Total HSSI TX PCSs	0 / 6 (0 %)
Total HSSI PMA TX Serializers	0 / 6 (0 %)
Total PLLs	0 / 13 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 1: Resource utilization of Direct FIR implementation (Non-pipelined)

Slow 1100mV 85C Model Fmax Summary					
 <<Filter>>					
Fmax	Restricted Fmax	Clock Name	Note		
51.9 MHz	51.9 MHz	clk1		1	

Figure 2: fmax of Direct FIR implementation (Non-pipelined)

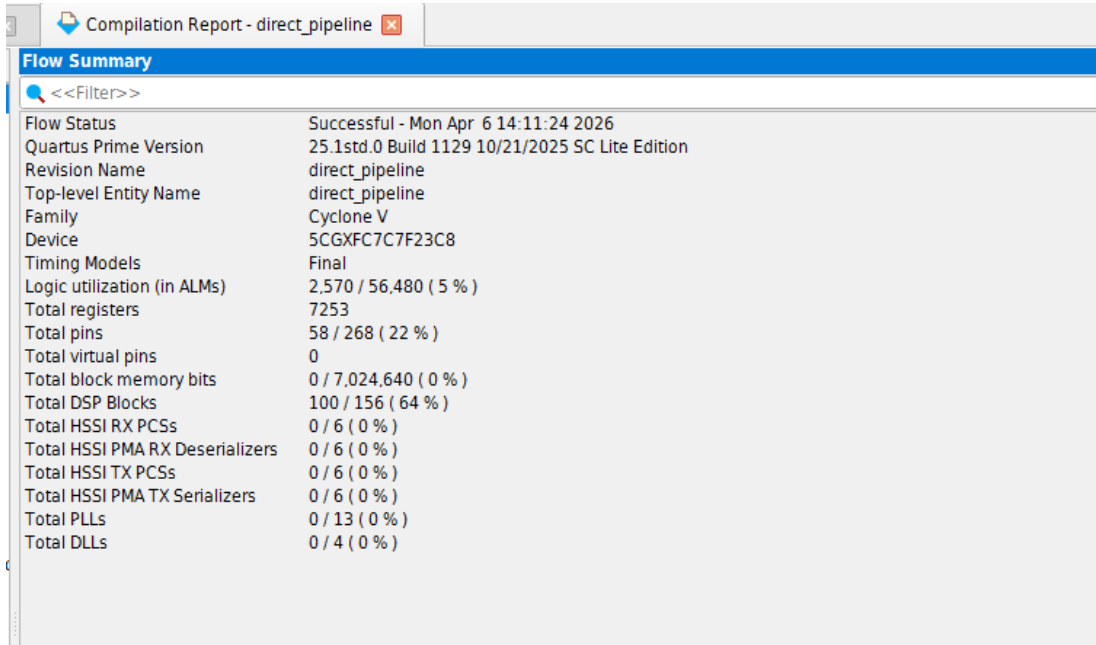


Figure 3: Resource utilization of Direct FIR implementation (Pipelined)

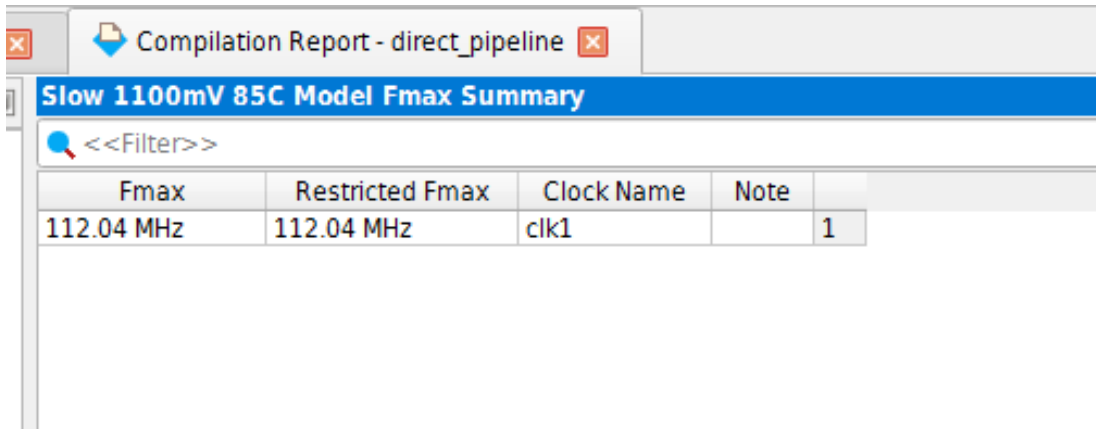


Figure 4: fmax of Direct FIR implementation (Pipelined)

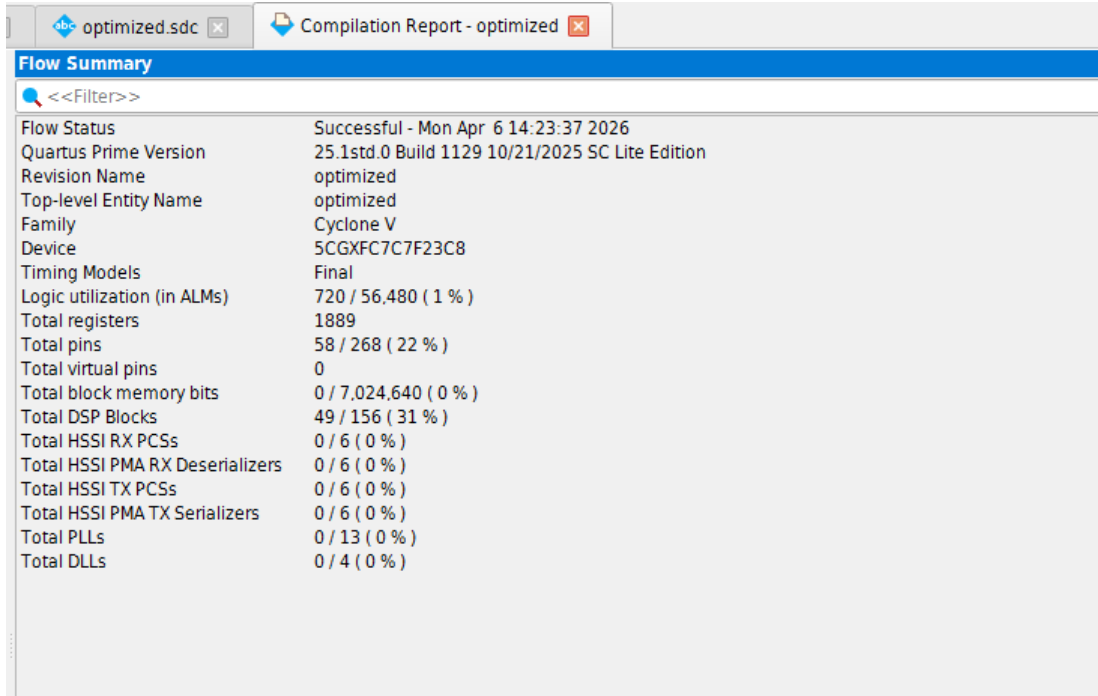


Figure 5: Resource utilization of Optimized FIR implementation (Non-pipelined)

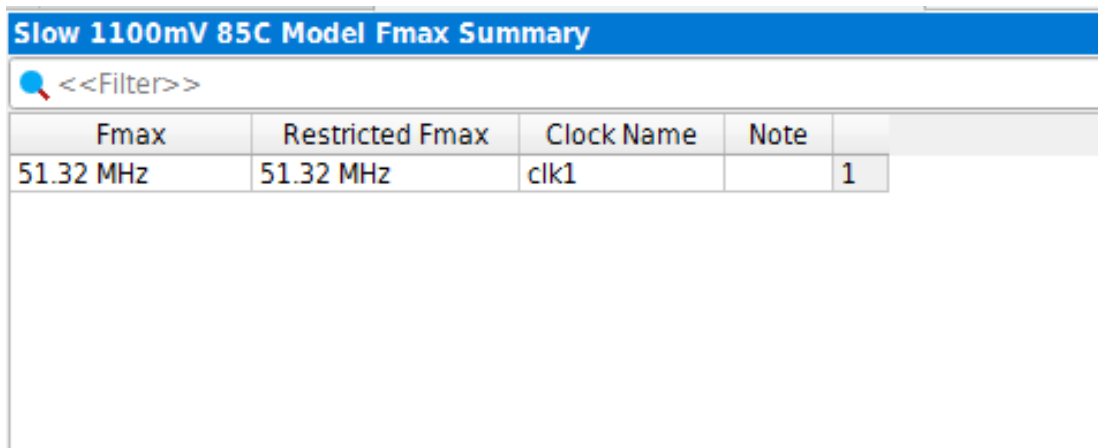


Figure 6: fmax of Optimized FIR implementation (Non-pipelined)

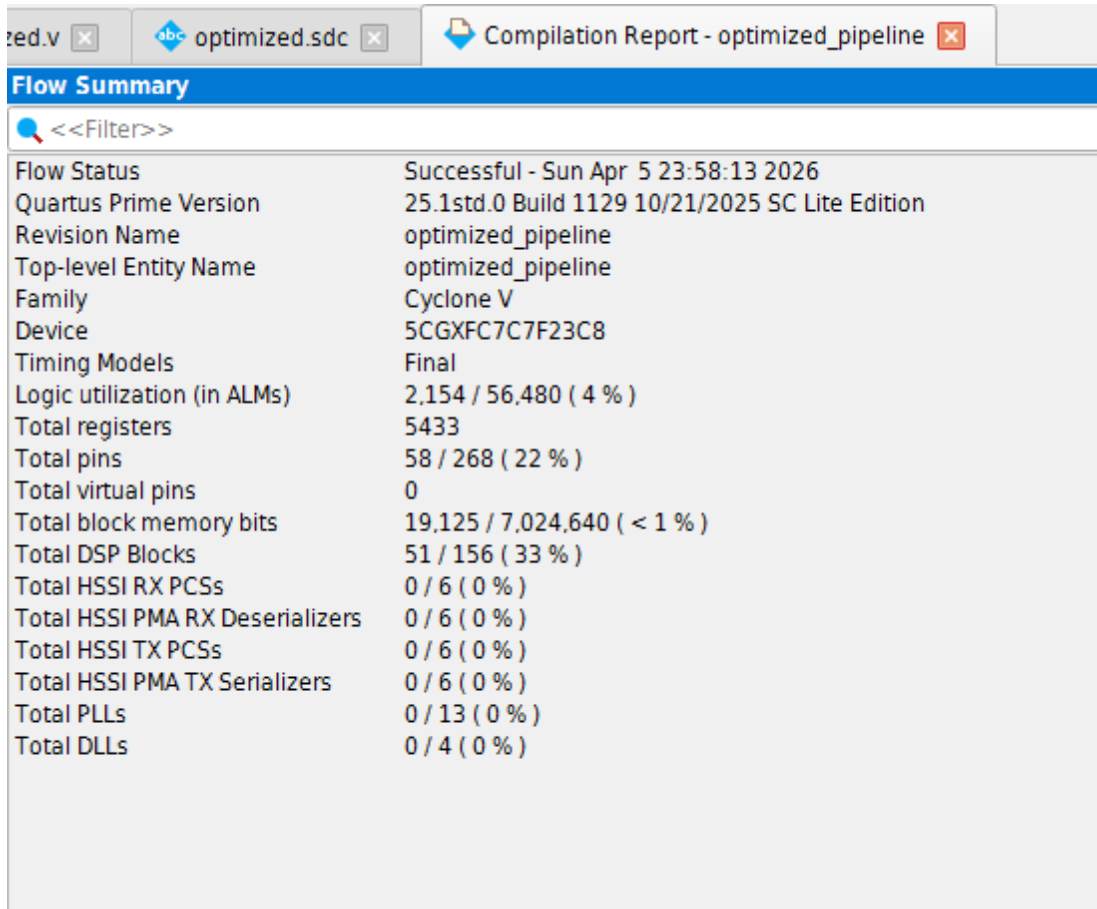


Figure 7: Resource utilization of Optimized FIR implementation (Pipelined)

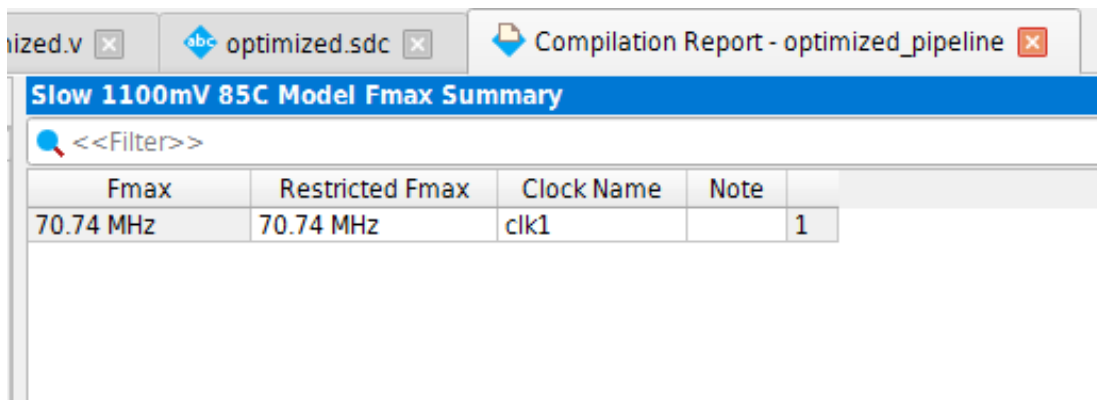


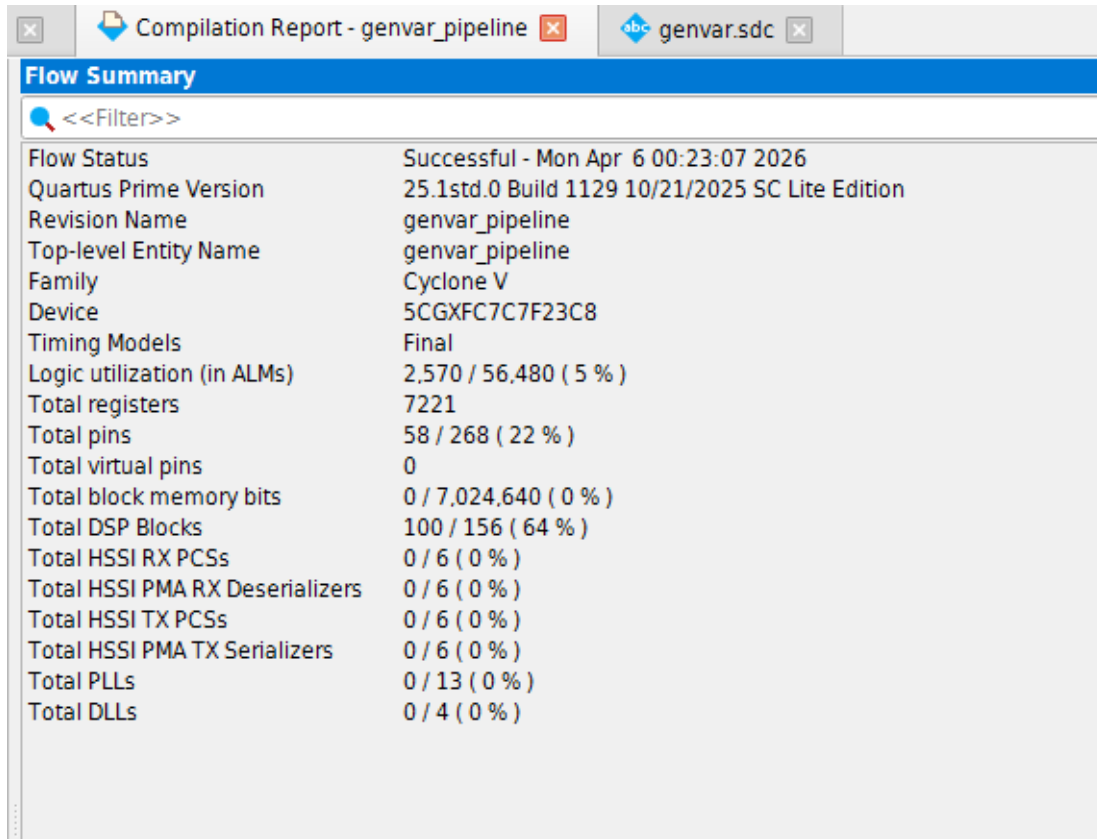
Figure 8: fmax of Optimized FIR implementation (Pipelined)

Flow Summary	
<<Filter>>	
Flow Status	Successful - Mon Apr 6 15:00:30 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	genvar
Top-level Entity Name	fir_genvar
Family	Cyclone V
Device	5CGXFC7C7F23C8
Timing Models	Final
Logic utilization (in ALMs)	1,094 / 56,480 (2 %)
Total registers	1879
Total pins	58 / 268 (22 %)
Total virtual pins	0
Total block memory bits	0 / 7,024,640 (0 %)
Total DSP Blocks	99 / 156 (63 %)
Total HSSI RX PCSs	0 / 6 (0 %)
Total HSSI PMA RX Deserializers	0 / 6 (0 %)
Total HSSI TX PCSs	0 / 6 (0 %)
Total HSSI PMA TX Serializers	0 / 6 (0 %)
Total PLLs	0 / 13 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 9: Resource utilization of Genvar FIR implementation (Non-pipelined)

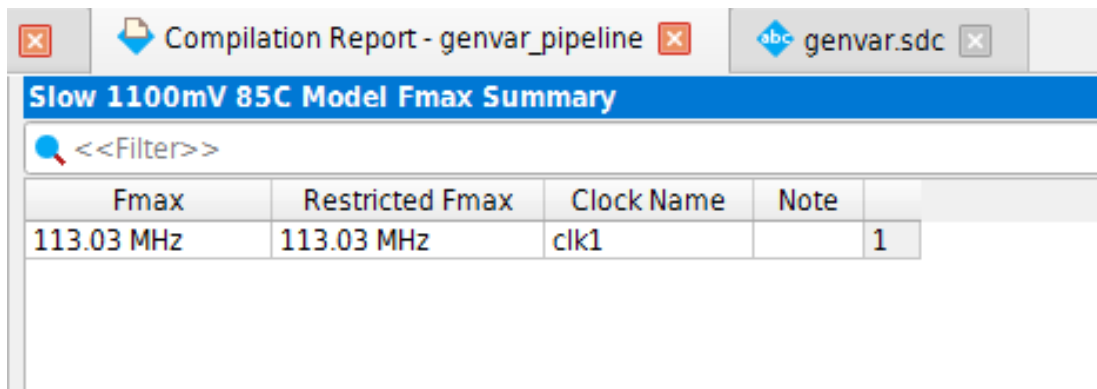
Slow 1100mV 85C Model Fmax Summary					
<<Filter>>					
Fmax	Restricted Fmax	Clock Name	Note		
53.89 MHz	53.89 MHz	clk1		1	

Figure 10: fmax of Genvar FIR implementation (Non-pipelined)



Flow Summary	
Flow Status	Successful - Mon Apr 6 00:23:07 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	genvar_pipeline
Top-level Entity Name	genvar_pipeline
Family	Cyclone V
Device	5CGXFC7C7F23C8
Timing Models	Final
Logic utilization (in ALMs)	2,570 / 56,480 (5 %)
Total registers	7221
Total pins	58 / 268 (22 %)
Total virtual pins	0
Total block memory bits	0 / 7,024,640 (0 %)
Total DSP Blocks	100 / 156 (64 %)
Total HSSI RX PCSs	0 / 6 (0 %)
Total HSSI PMA RX Deserializers	0 / 6 (0 %)
Total HSSI TX PCSs	0 / 6 (0 %)
Total HSSI PMA TX Serializers	0 / 6 (0 %)
Total PLLs	0 / 13 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 11: Resource utilization of Genvar FIR implementation (Pipelined)



Slow 1100mV 85C Model Fmax Summary				
Fmax	Restricted Fmax	Clock Name	Note	
113.03 MHz	113.03 MHz	clk1		1

Figure 12: fmax of Genvar FIR implementation (Pipelined)

Resource Utilization and Fmax Comparison

Implementation	Logic Utilization	Registers	DSP Blocks	Fmax (MHz)
Direct (Non-pipeline)	2%	1902	99	51.90
Direct (Pipeline)	5%	7253	100	112.04
Optimized (Non-pipeline)	1%	1889	49	51.32
Optimized (Pipeline)	4%	5433	51	70.74
Genvar (Non-pipeline)	2%	1879	99	53.89
Genvar (Pipeline)	5%	7221	100	113.03

Table 1: Resource utilization and maximum operating frequency comparison

Observation

From the resource utilization results, pipelined implementations require significantly more registers compared to non-pipelined designs because of the inserted pipeline stages. However, pipelining improves the operating speed of the filter. The optimized implementation reduces the number of multipliers by exploiting coefficient symmetry, making it more resource efficient. The genvar implementation simplifies the code structure while producing similar hardware usage as the direct implementation.

5 Output Comparison

The outputs obtained from MATLAB and Verilog implementations were compared for different input frequencies (950 Hz, 1100 Hz, and 2000 Hz). The plots were generated using MATLAB to verify the correctness of the hardware implementations.

For the Direct and Genvar pipeline implementations, the output appears after approximately 100 clock cycles of delay because the filter contains 100 taps. In the Optimized implementation, coefficient symmetry is used, which reduces the number of effective stages and results in a delay of approximately 50 clock cycles. Apart from this latency, the output waveforms match closely with the MATLAB results.

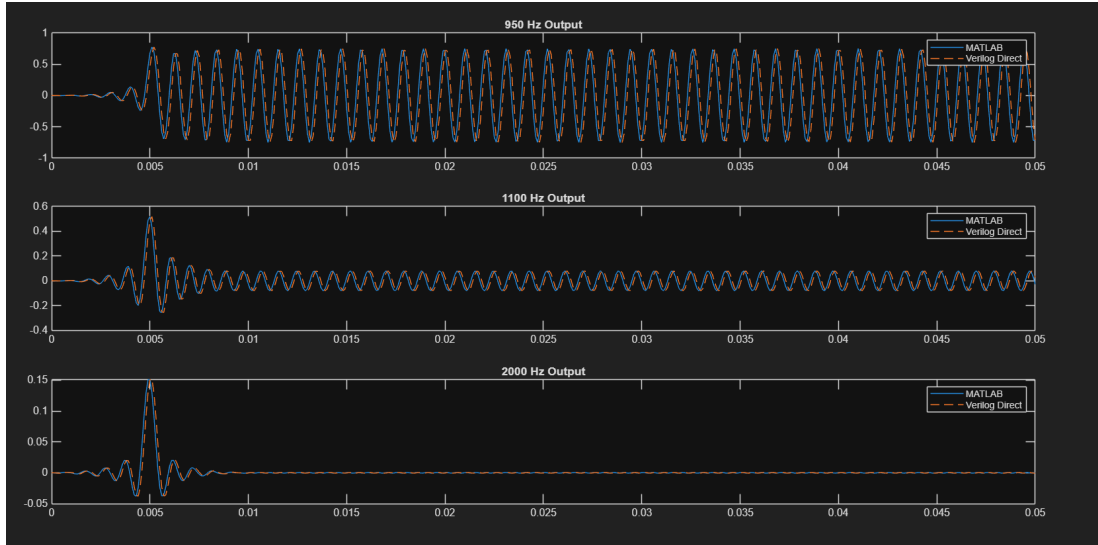


Figure 13: MATLAB vs Verilog Direct FIR implementation

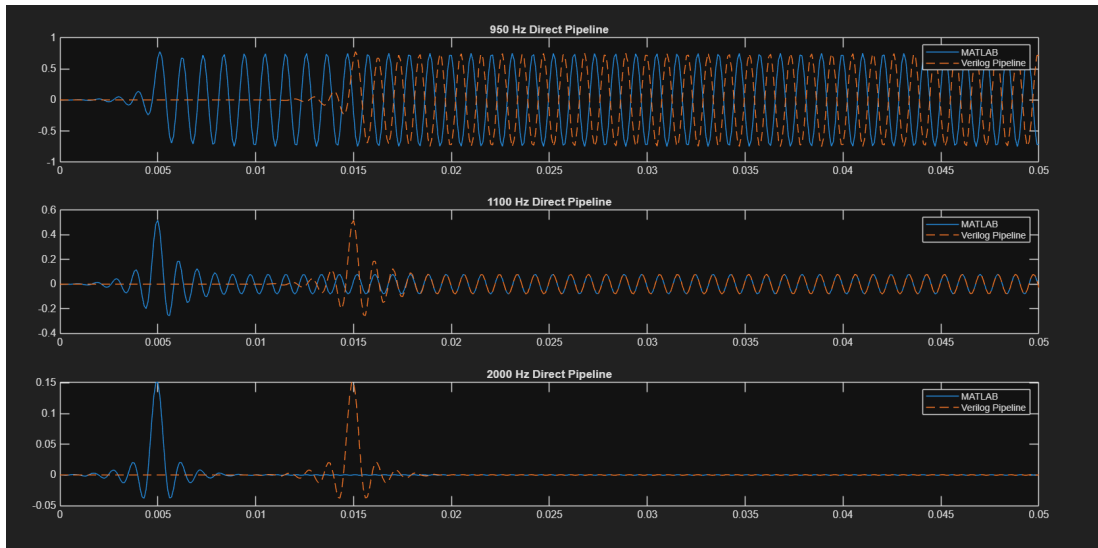


Figure 14: MATLAB vs Verilog Direct Pipeline FIR implementation (100 sample delay)

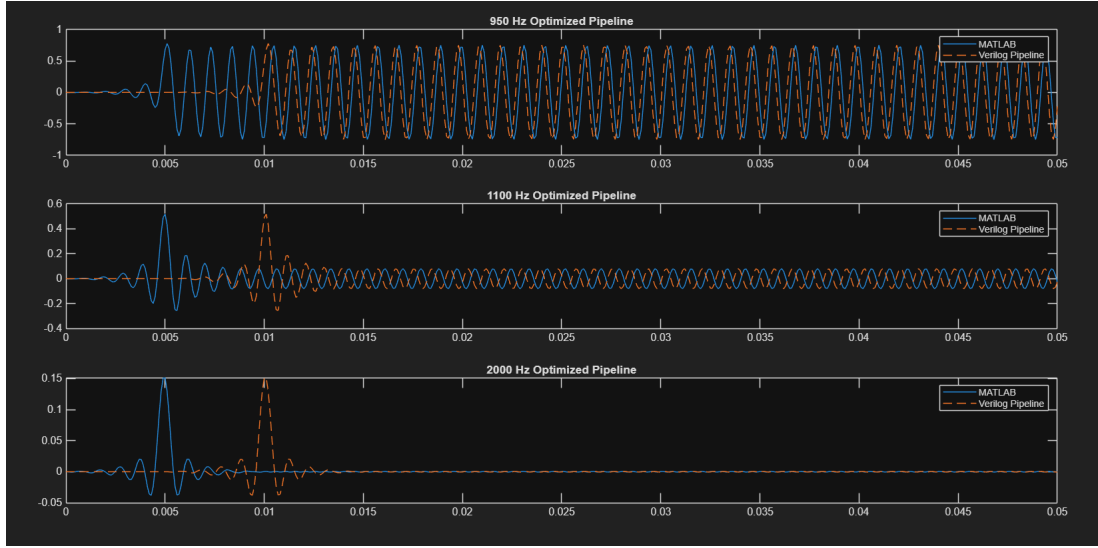


Figure 15: MATLAB vs Verilog Optimized Pipeline FIR implementation (50 sample delay)

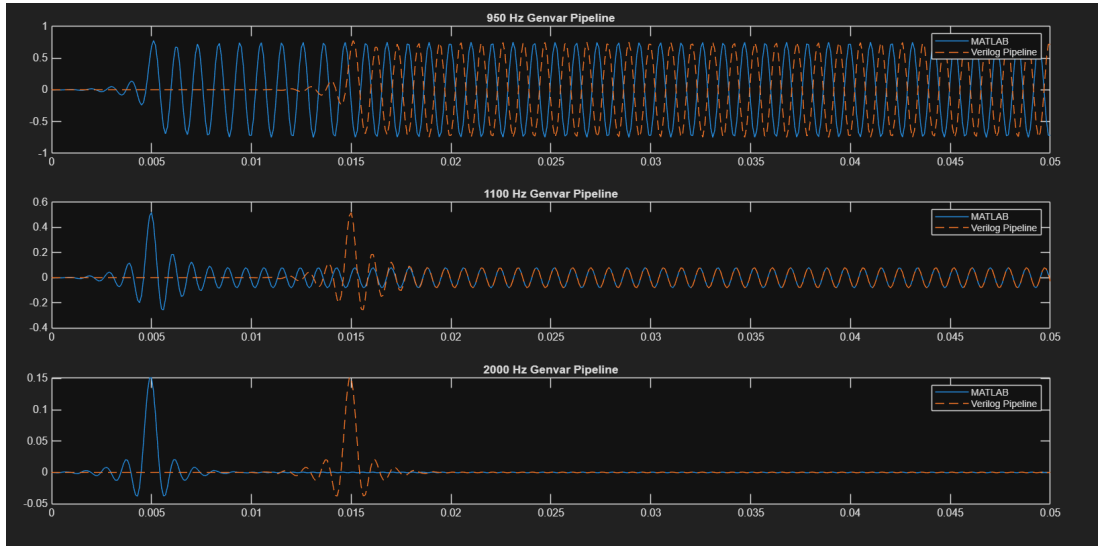


Figure 16: MATLAB vs Verilog Genvar Pipeline FIR implementation (100 sample delay)

From the plots, it can be observed that all implementations produce the same filtered output as the MATLAB reference model. The only difference is the pipeline latency introduced by the hardware architecture.

6 FIR Filter Using IP Core

Description

The FIR filter was also implemented using the FIR IP core provided in Quartus. The IP core automatically generates an optimized hardware architecture based on the given parameters such as number of taps, coefficient values, and data width. This approach reduces manual coding effort and provides an efficient hardware implementation. The IP core internally uses optimized pipelining and resource allocation techniques to achieve higher performance.

Resource Utilization

Flow Summary	
<<Filter>>	
Flow Status	Successful - Mon Apr 6 15:13:36 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	ip
Top-level Entity Name	fir_ip
Family	Arria II GX
Logic utilization	17 %
Total registers	6269
Total pins	63 / 176 (36 %)
Total virtual pins	0
Total block memory bits	0 / 2,939,904 (0 %)
DSP block 18-bit elements	10 / 232 (4 %)
Total GXB Receiver Channel PCS	0 / 4 (0 %)
Total GXB Receiver Channel PMA	0 / 4 (0 %)
Total GXB Transmitter Channel PCS	0 / 4 (0 %)
Total GXB Transmitter Channel PMA	0 / 4 (0 %)
Total PLLs	0 / 4 (0 %)
Total DLLs	0 / 2 (0 %)
Device	EP2AGX45CU17I3
Timing Models	Final

Figure 17: Resource utilization report for FIR filter implemented using IP core

Slow 900mV 100C Model Fmax Summary				
<<Filter>>				
	Fmax	Restricted Fmax	Clock Name	Note
1	166.78 MHz	166.78 MHz	clk1	

Figure 18: fmax for FIR filter implemented using IP core

Comparison with User Defined Modules

Implementation	Logic Utilization	Registers	DSP Blocks	Fmax (MHz)
Direct (Pipeline)	5%	7253	100	112.04
Optimized (Pipeline)	4%	5433	51	70.74
Genvar (Pipeline)	5%	7221	100	113.03
FIR IP Core	17%	6269	10	166.78

Table 2: Comparison of FIR IP core with user-defined implementations

Observation

From the comparison results, it can be observed that the FIR IP core achieves the highest maximum operating frequency of 166.78 MHz compared to the user-defined implementations. This is due to the optimized architecture generated automatically by the Quartus IP core.

The Direct and Genvar pipeline implementations achieve similar Fmax values around 112–113 MHz, while the Optimized pipeline implementation operates at a lower frequency due to additional logic used for coefficient symmetry. The IP core also uses fewer DSP blocks compared to some user-defined designs while still achieving higher performance.

Thus, the FIR IP core provides better speed performance, while user-defined modules provide flexibility for custom architectural design and optimization.

Observation

From the results, it can be observed that the FIR IP implementation uses optimized FPGA resources while requiring fewer registers compared to some user-defined implementations. The IP core automatically optimizes the architecture and simplifies the design process. User-defined modules provide flexibility in architecture design, whereas the IP core offers a faster and more structured method for implementing FIR filters.